Module 5 - SQL Data Definition Language (DDI)

Creating Tables, Keys, and Constraints

Recap & Transition

- Last week we normalized schemas (up to BCNF).
- Now we turn designs into implementation.

SQL Language Families

DDL: **D**ata **D**efinition **L**anguage

- Create & alter schemas

DML: **D**ata **M**anipulation **L**anguage

- Manipulate date (insert/update/delete)

DCL: **D**ata **C**ontrol **L**anguage

- Permissions & access

TCL: **T**ransaction **C**ontrol **L**anguage

- Transactions

TABLE 7.2

SQL DATA DEFINITION COMMANDS

COMMAND OR OPTION	DESCRIPTION	COVERED
CREATE SCHEMA AUTHORIZATION	**Creates a database schema**	**Chapter 8**
CREATE TABLE	**Creates a new table in the user's database schema**	**Chapter 8**
NOT NULL	Ensures that a column will not have null values	Chapter 8
UNIQUE	Ensures that a column will not have duplicate values	Chapter 8
PRIMARY KEY	Defines a primary key for a table	Chapter 8
FOREIGN KEY	Defines a foreign key for a table	Chapter 8
DEFAULT	Defines a default value for a column (when no value is given)	Chapter 8
CHECK	Validates data in an attribute	Chapter 8
CREATE INDEX	**Creates an index for a table**	**Chapter 8**
CREATE VIEW	**Creates a dynamic subset of rows and columns from one or more tables**	**Chapter 8**
ALTER TABLE	**Modifies a table's definition (adds, modifies, or deletes attributes or constraints)**	**Chapter 8**
CREATE TABLE AS	**Creates a new table based on a query in the user's database schema**	**Chapter 8**
DROP TABLE	**Permanently deletes a table (and its data)**	**Chapter 8**
DROP INDEX	**Permanently deletes an index**	**Chapter 8**
DROP VIEW	**Permanently deletes a view**	**Chapter 8**

Data Definition Commands (DDL)

1. Create a **database**
2. Determine attribute **data types**
3. Create **tables**
4. **Alter** table structures
5. **Delete** tables

Creating a Database

```
CREATE DATABASE projectdb;
\c projectdb   -- connect to the new database
```

Creating Tables

SQL Syntax

It is important to learn how to read and understand the syntax!

CAPITALS	Required SQL command **keywords**
Italics	A parameter provided **by the end user** (generally required)
{a | b | ..}	A **mandatory parameter**; use one option from the list
[.....]	An **optional parameter**--anything inside is optional
***T**ablename* / ***C**olumn*	
data type	A valid **data-type definition**
constraint	A valid **constraint definition**
condition	A valid **conditional expression** (evaluate to true or false)
columnlist	One or more column names or expressions **separated by commas**
tablelist	One or more table names separated by commas
conditionlist	One or more conditional expressions **separated by logical operators**
expression	A **simple value or a formula**

Creating Tables

```
CREATE TABLE tablename (
Column1      data type      [constraint] [,
Column2      data type      [constraint]] [,
PRIMARY KEY(column1    [, column2]) ] [,
FOREIGN KEY(column1    [, column2]) REFERENCES tablename][,
CONSTRAINT constraint] );
```

Creating Tables

CREATE TABLE *tablename* (

Column1 *data type* [*constraint*] [,

Column2 *data type* [*constraint*]] [,

PRIMARY KEY(*column1* [, *column2*])] [,

FOREIGN KEY(*column1* [, *column2*]) REFERENCES *tablename*][,

CONSTRAINT *constraint*]);

What is the **shortest possible** CREATE TABLE query?!

SQL **Data Types**

SQL Data Types

Common types:

- Character
  - Fixed length or variable Length
  - `CHAR` / `VARCHAR`
- Numeric
  - Different numeric types based on length and whether values can contain decimal digits
  - `NUMERIC`, `INTEGER`
- Date
  - Date arithmetics
  - `DATE`
- Boolean
  - `BOOLEAN`

Best practice: choose by meaning (e.g., area code = CHAR).

TABLE 8.1

SOME COMMON SQL DATA TYPES

DATA TYPE	FORMAT	COMMENTS
Numeric	NUMBER(L,D) or NUMERIC(L,D)	The declaration NUMBER(7,2) or NUMERIC(7,2) indicates that numbers will be stored with two decimal places and may be up to seven digits long, including the sign and the decimal place (for example, 12.32 or –134.99).
	INTEGER	May be abbreviated as INT. Integers are (whole) counting numbers, so they cannot be used if you want to store numbers that require decimal places.
	SMALLINT	Like INTEGER but limited to integer values up to six digits. If your integer values are relatively small, use SMALLINT instead of INT.
	DECIMAL(L,D)	Like the NUMBER specification, but the storage length is a *minimum* specification. That is, greater lengths are acceptable, but smaller ones are not. DECIMAL(9,2), DECIMAL(9), and DECIMAL are all acceptable.
Character	CHAR(L)	Fixed-length character data for up to 255 characters. If you store strings that are not as long as the CHAR parameter value, the remaining spaces are left unused. Therefore, if you specify CHAR(25), strings such as *Smith* and *Katzenjammer* are each stored as 25 characters. However, a U.S. area code is always three digits long, so CHAR(3) would be appropriate if you wanted to store such codes.
	VARCHAR(L) or VARCHAR2(L)	Variable-length character data. The designation VARCHAR2(25) or VARCHAR(25) will let you store characters up to 25 characters long. However, unlike CHAR, VARCHAR will not leave unused spaces. Oracle automatically converts VARCHAR to VARCHAR2.
Date	DATE	Stores dates in the Julian date format.

Determining Attribute Data Types

We must consider the **expected use of the attribute** to properly define attribute types.

Sometimes you need **professional judgment** to determine a data type.

TABLE 8.2

DATA DICTIONARY FOR THE CH08_SALECO DATABASE

TABLE NAME	ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK OR FK	FK REFERENCED TABLE
PRODUCT	P_CODE	Product code	VARCHAR(10)	XXXXXXXXXX	NA	Y	PK	
	P_DESCRIPT	Product description	VARCHAR(35)	Xxxxxxxxxxxx	NA	Y		
	P_INDATE	Stocking date	DATE	DD-MON-YYYY	NA	Y		
	P_QOH	Units available	SMALLINT	####	0–9999	Y		
	P_MIN	Minimum units	SMALLINT	####	0–9999	Y		
	P_PRICE	Product price	NUMBER(8,2)	####.##	0.00–9999.00	Y		
	P_DISCOUNT	Discount rate	NUMBER(5,2)	0.##	0.00–0.20	Y		
	V_CODE	Vendor code	INTEGER	###	100–999		FK	VENDOR
VENDOR	V_CODE	Vendor code	INTEGER	#####	1000–9999	Y	PK	
	V_NAME	Vendor name	VARCHAR(35)	Xxxxxxxxxxxxxx	NA	Y		
	V_CONTACT	Contact person	VARCHAR(25)	Xxxxxxxxxxxxxx	NA	Y		
	V_AREACODE	Area code	CHAR(3)	999	NA	Y		
	V_PHONE	Phone number	CHAR(8)	999–9999	NA	Y		
	V_STATE	State	CHAR(2)	XX	NA	Y		
	V_ORDER	Previous order	CHAR(1)	X	Y or N	Y		

FK = Foreign key
PK = Primary key
CHAR = Fixed-length character data, 1 to 255 characters
VARCHAR = Variable-length character data, 1 to 2,000 characters. VARCHAR is automatically converted to VARCHAR2 in Oracle.
NUMBER = Numeric data. NUMBER(9,2) is used to specify numbers that have two decimal places and are up to nine digits long, including the decimal places. Some RDBMSs permit the use of a MONEY or a CURRENCY data type.
NUMERIC = Numeric data. DBMSs that do not support the NUMBER data type typically use NUMERIC instead.
INT = Integer values only. INT is automatically converted to NUMBER in Oracle.
SMALLINT = Small integer values only. SMALLINT is automatically converted to NUMBER in Oracle.
DATE formats vary. Commonly accepted formats are DD-MON-YYYY, DD-MON-YY, MM/DD/YYYY, and MM/DD/YY.

*Not all the ranges shown here will be illustrated in this chapter. However, you can use these constraints to practice writing your own.

Example

V_AREACODE is **saved as a character (CHAR(3))** even though it only contains digits - **why?**

1. Adding / subtracting area codes does not make sense!
2. An area/zip code might start with a zero. If saved as a number, the leading zero might get dropped.

Example

If you save the number of bathrooms (**H_BATH_NUM**) as CHAR(3) -- you might think that the application should not do arithmetics (+, -, /, *) with the number -- then:

A query to sort the homes by number of bathrooms would put '10' before '2'!

Here's where you need to use your **professional judgment** to decide whether H_BATH_NUM should be stored as character or numeric.

SQL Constraints

PRIMARY KEY

FOREIGN KEY

Actions: ON DELETE / ON UPDATE → CASCADE / SET NULL / SET DEFAULT

NOT NULL

UNIQUE

DEFAULT

CHECK

To **validate** the value entered for an attribute

Vendor Table

```
CREATE TABLE VENDOR (
V_CODE        INTEGER       NOT NULL      UNIQUE,
V_NAME        VARCHAR(35)   NOT NULL,
V_CONTACT     VARCHAR(25)   NOT NULL,
V_AREACODE    CHAR(3)       NOT NULL,
V_PHONE       CHAR(8)       NOT NULL,
V_STATE       CHAR(2)       NOT NULL,
V_ORDER       CHAR(1)       NOT NULL,
PRIMARY KEY(V_CODE));
```

Vendor Table

```
CREATE TABLE VENDOR (
V_CODE        INTEGER      NOT NULL    UNIQUE,
V_NAME        VARCHAR(35)  NOT NULL,
V_CONTACT     VARCHAR(25)  NOT NULL,
V_AREACODE    CHAR(3)      NOT NULL,
V_PHONE       CHAR(8)      NOT NULL,
V_STATE       CHAR(2)      NOT NULL,
V_ORDER       CHAR(1)      NOT NULL,
PRIMARY KEY(V_CODE));
```

data type

constraint

Spaces are used to **line up** pieces of a query

Product Table

```
CREATE TABLE PRODUCT (
P_CODE        VARCHAR(10)     NOT NULL     UNIQUE,
P_DESCRIPT    VARCHAR(35)     NOT NULL,
P_INDATE      DATE            NOT NULL,
P_QOH         SMALLINT        NOT NULL,
P_MIN         SMALLINT        NOT NULL,
P_PRICE       NUMERIC(8,2)    NOT NULL,
P_DISCOUNT    NUMERIC(5,2)    NOT NULL,
V_CODE        INTEGER
PRIMARY KEY(P_CODE),
FOREIGN KEY(V_CODE) REFERENCES VENDOR ON UPDATE CASCADE );
```

Product Table

```
CREATE TABLE PRODUCT (
P_CODE        VARCHAR(10)    NOT NULL    UNIQUE,
P_DESCRIPT    VARCHAR(35)    NOT NULL,
P_INDATE      DATE           NOT NULL,
P_QOH         SMALLINT       NOT NULL,
P_MIN         SMALLINT       NOT NULL,
P_PRICE       NUMERIC(8,2)   NOT NULL,
P_DISCOUNT    NUMERIC(5,2)   NOT NULL
V_CODE        INTEGER
PRIMARY KEY(P_CODE),
FOREIGN KEY(V_CODE) REFERENCES VENDOR ON UPDATE CASCADE );
```

Ensures **entity integrity**

- Creates a **unique index** on *P_CODE*
- Use UNIQUE to **avoid duplicate values** in a column

- Ensures **referential integrity**
- If *VENDOR*'s *V_CODE* is changed, all FKs referencing the *V_CODE* are updated.

Customer Table

```
CREATE TABLE CUSTOMER (

CUS_CODE          NUMERIC        PRIMARY KEY,

CUS_LNAME         VARCHAR(15)    NOT NULL,

CUS_FNAME         VARCHAR(15)    NOT NULL,

CUS_INITIAL       CHAR(1),

CUS_AREACODE      CHAR(3)        DEFAULT '615'   NOT NULL

                                 CHECK(CUS_AREACODE IN ('615', '713', '931')),

CUS_PHONE         CHAR(8)        NOT NULL,

CUS_BALANCE       NUMERIC(9, 2)  DEFAULT 0.00,

CONSTRAINT    CUS_UI1   UNIQUE(CUS_LNAME, CUS_FNAME));
```

Customer Table

```
CREATE TABLE CUSTOMER (
CUS_CODE          NUMERIC        PRIMARY KEY,
CUS_LNAME         VARCHAR(15)    NOT NULL,
CUS_FNAME         VARCHAR(15)    NOT NULL,
CUS_INITIAL       CHAR(1),
CUS_AREACODE      CHAR(3)        DEFAULT '615'   NOT NULL
                                 CHECK(CUS_AREACODE IN ('615', '713', '931')),
CUS_PHONE         CHAR(8)        NOT NULL,
CUS_BALANCE       NUMERIC(9, 2)  DEFAULT 0.00,
CONSTRAINT  CUS_UI1  UNIQUE(CUS_LNAME, CUS_FNAME));
```

Constraints can be defined in two places:

1. In column definition (**column constraint**) → applies to one column
2. Using CONSTRAINT keyword (**table constraint**) → may apply to multiple columns

Awayse **name your constraints**

CHECK only validates values for attributes in this table. To validate attributes in other tables you should use **triggers**

Invoice Table

```
CREATE TABLE INVOICE (
INV_NUMBER      NUMERIC     PRIMARY KEY,
CUS_CODE        NUMERIC     NOT NULL     REFERENCES CUSTOMER(CUS_CODE),
INV_DATE        DATE        DEFAULT SYSDATETIME()  NOT NULL,
CONSTRAINT   INV_CK1   CHECK(INV_DATE >'01-JAN-2018'));
```

Invoice Table

```
CREATE TABLE INVOICE (

INV_NUMBER      NUMERIC     PRIMARY KEY,

CUS_CODE        NUMERIC     NOT NULL     REFERENCES CUSTOMER(CUS_CODE),

INV_DATE        DATE        DEFAULT SYSDATETIME()  NOT NULL,

CONSTRAINT   INV_CK1   CHECK(INV_DATE >'01-JAN-2018'));
```

You see that a **FK constraint** can also be defined in column definition. Remember:

Constraints can be defined in column definition or with CONSTRAINT keyword

Line Table

```
CREATE TABLE LINE (
INV_NUMBER        NUMERIC        NOT NULL,
LINE_NUMBER       NUMERIC(2, 0)  NOT NULL,
P_CODE            VARCHAR(10)    NOT NULL,
LINE_UNITS        NUMERIC(9, 2)  DEFAULT 0.00   NOT NULL ,
LINE_PRICE        NUMERIC(9, 2)  DEFAULT 0.00   NOT NULL ,
PRIMARY KEY(INV_NUMBER, LINE_NUMBER),
FOREIGN KEY(INV_NUMBER) REFERENCES INVOICE ON DELETE CASCADE,
FOREIGN KEY(P_CODE) REFERENCES PRODUCT(P_CODE),
CONSTRAINT   LINE_UI1   UNIQUE(INV_NUMBER, P_CODE));
```

Line Table

```
CREATE TABLE LINE (

INV_NUMBER        NUMERIC        NOT NULL,

LINE_NUMBER       NUMERIC(2, 0)  NOT NULL,

P_CODE            VARCHAR(10)    NOT NULL,

LINE_UNITS        NUMERIC(9, 2)  DEFAULT 0.00   NOT NULL ,

LINE_PRICE        NUMERIC(9, 2)  DEFAULT 0.00   NOT NULL ,

PRIMARY KEY(INV_NUMBER, LINE_NUMBER),

FOREIGN KEY(INV_NUMBER) REFERENCES INVOICE ON DELETE CASCADE,

FOREIGN KEY(P_CODE) REFERENCES PRODUCT(P_CODE),

CONSTRAINT    LINE_UI1   UNIQUE(INV_NUMBER, P_CODE));
```

Composite PK → should be defined as table constraint

Recommended for **weak entities**

To ensure the same product is not ordered twice in the same invoice

A Few Notes

RDMBS automatically enforces referential integrity

- FKs cannot be invalid
- Parent record cannot be deleted if referenced by a child

The **order** in which you create tables **matters**

- VENDOR should be created before PRODUCT.
- Alwayse create the 1 side of a 1:M relationship first. **Why?**

Altering Table Structures

Altering & Dropping Tables

You can use ALTER TABLE command to:

- Add / Modify columns
- Add constraints
- Remove a column / constraint
- Change a column data type or other characteristics

You can use DROP TABLE command to delete a table from the database

Checklist for Good Schema DDL

- ❑ Primary Key defined.
- ❑ Foreign Keys enforce relationships.
- ❑ Data types match semantics.
- ❑ Constraints enforce business rules.

Takeaways

- DDL turns designs into actual databases.
- Constraints protect integrity.
- Sets up next week: DML (working with data).